UNIVERSIDAD NACIONAL DEL ALTIPLANO
Escuela Profesional de Ingeniería de Sistemas

INFORME DE PRÁCTICA DE LABORATORIO

PRÁCTICA CALIFICADA — SEMANA 3

Herencia y Polimorfismo

Jerarquías, sobrescritura y Principio de Sustitución de Liskov

Estudiante	Kevin Manuel Vilca Paco
Docente	Dr. Aldo Hernán Zanabria Gálvez
Curso	SIS206 — Programación Orientada a Objetos II
Semestre	2026-II
Semana / Práctica	Semana 3 — Práctica Calificada (PC1)
Lenguajes	Python 3.12+ · C++17

Puno — Perú
2026

Contenido

1. Introducción y objetivos
2. Marco teórico resumido
3. Actividad 1 — Análisis IS-A vs HAS-A
4. Actividad 2 — Jerarquía de Personal en Python
5. Actividad 3 — Verificación del Principio de Liskov (LSP)
6. Actividad 4 — Jerarquía en C++17 (virtual, override, destructor)
7. Actividad 5 — Proyecto Integrador: Sistema de Biblioteca
8. Preguntas de reflexión
9. Conclusiones
10. Referencias

Anexo — Estructura del repositorio y comandos de ejecución

> **¿Cómo leer este informe?**
> Cada actividad sigue el mismo orden: **qué se pide → cómo se resolvió (código) → UML → salida real del programa → análisis corto**.
> Todas las salidas mostradas fueron generadas ejecutando los programas incluidos en el repositorio (carpeta `salidas/`).

1. Introducción y objetivos

El presente informe documenta el desarrollo de la **Práctica Calificada de la Semana 3** del curso Programación Orientada a Objetos II. La práctica trata sobre **herencia y polimorfismo**: cómo organizar clases en jerarquías bien justificadas, cómo sobrescribir métodos y cómo garantizar que cualquier subclase pueda reemplazar a su clase base sin romper el programa (Principio de Sustitución de Liskov).

1.1 Objetivo general

Diseñar e implementar jerarquías de herencia correctamente justificadas en **Python** y **C++17**, aplicando sobrescritura de métodos, polimorfismo dinámico y el principio de sustitución de Liskov (LSP), avanzando en el Proyecto Integrador del semestre.

1.2 Objetivos específicos

- Identificar cuándo la herencia está justificada (**IS-A**) y cuándo conviene composición (**HAS-A**).
- Implementar jerarquías con `super()` en Python y con herencia pública y `virtual` en C++.
- Sobrescribir métodos con `@override` (Python 3.12+) y `override` (C++17), entendiendo la *vtable*.
- Verificar el LSP: todo objeto de una subclase debe poder sustituir a su clase base.
- Extender el Proyecto Integrador con una jerarquía de herencia justificada.

1.3 Herramientas utilizadas

Herramienta	Versión	Uso
Python	3.12 (`typing.override`)	Actividades 2, 3 y 5
g++	13.3 — `-std=c++17 -Wall -Wextra -pedantic`	Actividades 4 y 5
Graphviz	`dot` (código UML en archivos `.dot`)	Diagramas de clases
Git / GitHub	Commit `feat(semana3): jerarquía de herencia`	Entrega del código

2. Marco teórico resumido

2.1 Herencia: IS-A frente a HAS-A

La herencia modela la relación **"ES UN"** (un `Docente` ES UN `PersonalUniversitario`). La composición modela **"TIENE UN"** (un `Docente` TIENE UN `Departamento`). Usar herencia cuando corresponde composición produce jerarquías frágiles (Larman, 2004).

	Herencia (IS-A)	Composición (HAS-A)
Pregunta clave	¿X es un Y en todos los contextos?	¿X tiene o usa un Y?
Ejemplo correcto	`Docente` → `PersonalUniversitario`	`Docente` ◆ `Departamento`
Ejemplo incorrecto	`Automovil(Motor)` ✗	—
Acoplamiento	Alto (la subclase depende de la base)	Bajo (se puede cambiar la parte)

2.2 Polimorfismo dinámico y sobrescritura

El polimorfismo dinámico permite que una referencia del tipo **base** invoque el método de la subclase **real** en tiempo de ejecución. En Python es automático; en C++ se necesita declarar el método `virtual` en la base y `override` en las subclases. El compilador construye una **vtable** (tabla de punteros a funciones) por clase, y cada objeto guarda un puntero oculto (*vptr*) a la vtable de su clase real.

2.3 Principio de Sustitución de Liskov (LSP)

Si **S** es subtipo de **T**, los objetos de tipo T pueden reemplazarse por objetos de tipo S sin alterar las propiedades deseables del programa (Liskov, 1987). En la práctica: una subclase **no debe debilitar las precondiciones ni fortalecer las poscondiciones** de la clase base.

> **Idea clave para recordar**
> Herencia ≠ "se parecen". Herencia = **"puedo usar la subclase en cualquier lugar donde se espera la base y todo sigue funcionando"**.

3. Actividad 1 — Análisis IS-A vs HAS-A

Qué se pide: para cada relación indicar si se implementa con herencia (IS-A) o composición (HAS-A) y justificar. Si hay herencia, nombrar la clase base y la subclase.

#	Relación	Decisión	Base → Subclase	Justificación
1	Estudiante de Posgrado / Estudiante	IS-A Herencia	`Estudiante` → `EstudiantePosgrado`	Un estudiante de posgrado *es un* estudiante: conserva código, matrícula y cursos; solo añade asesor, tesis y línea de investigación.
2	Sensor IoT / Dispositivo	IS-A Herencia	`Dispositivo` → `SensorIoT`	Un sensor *es un* dispositivo: hereda id, estado y encender/apagar; agrega lectura, unidad y frecuencia de muestreo.
3	Informe / Impresora	HAS-A Asociación	—	Un informe no es una impresora. La impresora *usa* un informe (`imprimir(informe)`). Heredar sería como `Automovil(Motor)`.
4	Libro Digital / Libro	IS-A Herencia (con cuidado)	`Libro` → `LibroDigital`	Es un libro (título, autor, ISBN). Cuidado con el LSP: la base no debe tener peso ni ejemplares físicos. Por eso en el proyecto se usó la base `RecursoBibliografico`.
5	Motocicleta / Vehículo	IS-A Herencia	`Vehiculo` → `Motocicleta`	Una moto *es un* vehículo: placa, marca, acelerar y frenar. Añade cilindrada. Relación taxonómica clásica y sustituible.
6	Departamento / Empleado	HAS-A Composición	—	Un departamento no es un empleado: *tiene* empleados (`Departamento ◇── Empleado`). Son conceptos distintos.

Análisis

Cuatro relaciones son herencia legítima (1, 2, 4, 5) y dos son composición/asociación (3, 6). El error típico es heredar solo porque las clases "se relacionan": relacionarse no es lo mismo que ser.

4. Actividad 2 — Jerarquía de Personal en Python

Qué se pide: completar las subclases `Docente` y `Administrativo` (validaciones + cálculo de remuneración) y escribir `planilla()`, que demuestre polimorfismo **sin usar isinstance()**. Archivo: `actividad2_python/personal_unap.py`.

4.1 Diseño (UML)

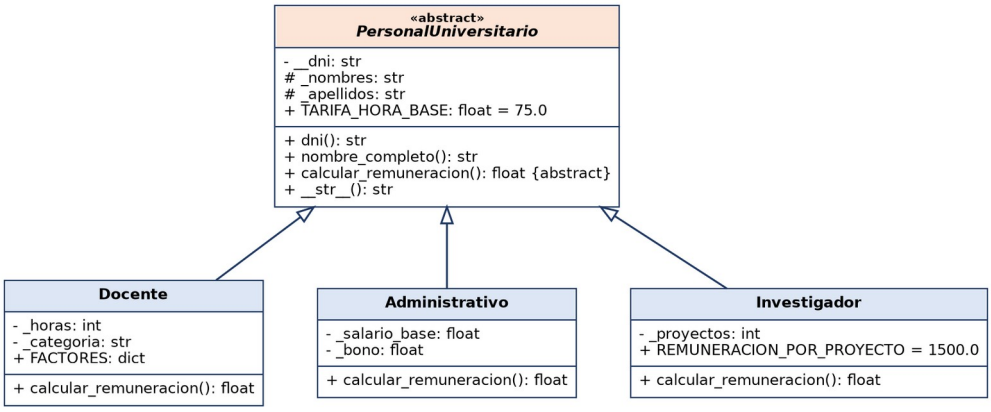


Figura 1. Jerarquía PersonalUniversitario (flecha hueca = herencia).

4.2 Implementación

Los `TODO` del enunciado se resolvieron así: `Docente` valida horas entre 1 y 40 y que la categoría exista en `FACTORES`; `Administrativo` valida que el salario base sea positivo. Ambas usan `super().__init__()` para reutilizar el constructor (y la validación de DNI) de la clase base, y `@override` para marcar la sobrescritura.

personal_unap.py — Docente y Administrativo (TODOs completados)

```
class Docente(PersonalUniversitario):
    FACTORES = {"Auxiliar": 1.0, "Asociado": 1.4, "Principal": 1.8}

    def __init__(self, dni: str, nombres: str, apellidos: str,
                 horas: int, categoria: str):
        super().__init__(dni, nombres, apellidos)
        # Validar horas > 0 y <= 40
        if not (0 < horas <= 40):
            raise ValueError(f"Horas fuera de rango: {horas} (debe estar en 1..40)")
        # Validar categoría en FACTORES
        if categoria not in self.FACTORES:
            raise ValueError(
                f"Categoría inválida: {categoria!r} "
                f"(use: {', '.join(self.FACTORES)})"
            )
        # Asignar atributos
        self._horas = horas
        self._categoria = categoria

    @override
    def calcular_remuneracion(self) -> float:
        # horas * TARIFA_HORA_BASE * factor de categoría
        return self._horas * self.TARIFA_HORA_BASE * self.FACTORES[self._categoria]


class Administrativo(PersonalUniversitario):
```

```
    def __init__(self, dni: str, nombres: str, apellidos: str,
                 salario_base: float, bono: float = 0.0):
        super().__init__(dni, nombres, apellidos)
        # Validar salario_base > 0
        if salario_base <= 0:
            raise ValueError("El salario base debe ser mayor que 0")
        if bono < 0:
            raise ValueError("El bono no puede ser negativo")
        # Asignar atributos
        self._salario_base = salario_base
        self._bono = bono

    @override
    def calcular_remuneracion(self) -> float:
        # salario_base + bono
        return self._salario_base + self._bono
```

personal_unap.py — planilla() polimórfica

```
def planilla(personal: list[PersonalUniversitario]) -> None:
    """Imprime la planilla usando polimorfismo — sin isinstance()."""
    total = 0.0
    print("=" * 60)
    print("PLANILLA PERSONAL UNIVERSITARIO — UNAP PUNO")
    print("=" * 60)
    for p in personal:
        print(p)  # llama __str__ → calcular_remuneracion polimórfico
        total += p.calcular_remuneracion()
    print("-" * 60)
    print(f"TOTAL PLANILLA: S/ {total:.2f}")
```

4.3 Salida del programa

$ python3 actividad2_python/personal_unap.py

```
============================================================
PLANILLA PERSONAL UNIVERSITARIO — UNAP PUNO
============================================================
Mamani Quispe, Juan Carlos → S/ 1680.00
Quispe Torres, María Elena → S/ 1620.00
Huanca Lazo, Pedro → S/ 3150.00
Flores Ramos, Ana Lucía → S/ 3000.00
------------------------------------------------------------
TOTAL PLANILLA: S/ 9450.00

Verificación LSP: OK — todos los objetos respetan el contrato base

PRUEBA DE VALIDACIONES
------------------------------------------------------------
[OK] DNI con 7 dígitos
     → ValueError: DNI inválido: '1234567' (debe tener 8 dígitos)
[OK] Horas = 50 (máx. 40)
     → ValueError: Horas fuera de rango: 50 (debe estar en 1..40)
[OK] Categoría inexistente
     → ValueError: Categoría inválida: 'Titular' (use: Auxiliar, Asociado, Principal)
[OK] Salario base negativo
     → ValueError: El salario base debe ser mayor que 0
[OK] Investigador sin proyectos
     → ValueError: Debe tener al menos 1 proyecto activo
[OK] Instanciar clase abstracta
     → TypeError: Can't instantiate abstract class PersonalUniversitario without an implementation
for abstract method 'calcular_remuneracion'
```

4.4 Análisis

- **Polimorfismo:** `planilla()` solo conoce `PersonalUniversitario`. Al hacer `print(p)` se llama `__str__`, que invoca `calcular_remuneracion()` y Python ejecuta la versión de la subclase real (Docente, Administrativo o Investigador).
- **Cálculos verificados a mano:** Mamani 16 × 75 × 1.4 = 1680; Quispe 12 × 75 × 1.8 = 1620; Huanca 2800 + 350 = 3150; Flores 2 × 1500 = 3000. **Total = S/ 9450.00** ✔️.
- **Validaciones:** las 6 pruebas de datos inválidos lanzan la excepción esperada; además la clase abstracta no se puede instanciar (`TypeError`).
- **Extensibilidad:** agregar un nuevo tipo de personal no obliga a modificar `planilla()` (principio abierto/cerrado).

5. Actividad 3 — Verificación del Principio de Liskov (LSP)

Qué se pide: (a) un contrato invariante que todas las subclases de `PersonalUniversitario` deban cumplir; (b) mostrar que `Cuadrado` hereda de `Rectangulo` y **viola** el LSP; (c) explicar por qué y proponer una alternativa. Archivo: `actividad3_lsp/lsp_verificacion.py`.

5.1 Parte A — Contrato sobre PersonalUniversitario

verificar_contrato_personal()

```
def verificar_contrato_personal(p: PersonalUniversitario) -> bool:
    """
    Contrato que TODA subclase debe satisfacer:
    1. nombre_completo retorna string no vacío
    2. calcular_remuneracion() retorna valor > 0
    3. str(p) contiene el nombre completo
    """
    ok = True
    ok = ok and isinstance(p.nombre_completo, str) and len(p.nombre_completo) > 0
    ok = ok and p.calcular_remuneracion() > 0
    ok = ok and p.nombre_completo in str(p)
    return ok
```

El contrato exige tres cosas a cualquier subclase: nombre completo no vacío, remuneración mayor que 0 y que `str(p)` contenga el nombre completo.

5.2 Parte B — Cuadrado hereda de Rectangulo (violación)

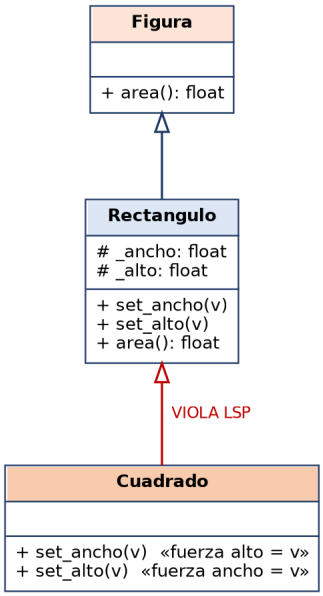


Figura 2. Cuadrado hereda de Rectangulo y cambia el contrato de los setters.

Cuadrado + test_rectangulo()

```
class Cuadrado(Rectangulo):
    def __init__(self, lado: float):
        super().__init__(lado, lado)

    def set_ancho(self, v: float) -> None:
        self._ancho = v
        self._alto = v  # fuerza ambos lados iguales
```

```
    def set_alto(self, v: float) -> None:
        self._ancho = v
        self._alto = v


def test_rectangulo(r: Rectangulo) -> None:
    """Test que DEBERÍA pasar para cualquier Rectangulo según LSP."""
    r.set_ancho(4)
    r.set_alto(5)
    area_esperada = 20.0
    assert r.area() == area_esperada, (
        f"LSP violado: area={r.area()}, esperado={area_esperada}"
    )
```

5.3 Salida del programa

```
$ python3 actividad3_lsp/lsp_verificacion.py
============================================================
PARTE A — Contrato LSP sobre PersonalUniversitario
============================================================
[OK ] Docente        Mamani Quispe, Juan Carlos
[OK ] Docente        Quispe Torres, María Elena
[OK ] Administrativo Huanca Lazo, Pedro
[OK ] Investigador   Flores Ramos, Ana Lucía
→ Todas las subclases superan el contrato: LSP respetado.

============================================================
PARTE B — Cuadrado hereda de Rectangulo (violación del LSP)
============================================================
[OK ] test_rectangulo(Rectangulo)  → área = 20.0
[FALLA] test_rectangulo(Cuadrado) → LSP violado: area=25, esperado=20.0
LSP VIOLADO en Cuadrado.
Solución: no heredar Cuadrado de Rectangulo; usar composición o interfaz independiente.

============================================================
PARTE C — Rediseño sin herencia Cuadrado→Rectangulo (cumple LSP)
============================================================
[OK ] RectanguloInmutable  área= 20.0  perímetro= 18.0
[OK ] CuadradoInmutable    área=  9.0  perímetro= 12.0
→ Cualquier FiguraGeometrica es sustituible por otra: LSP respetado.
```

5.4 Respuesta a la pregunta de análisis

¿Por qué Cuadrado viola el LSP aunque matemáticamente un cuadrado ES un rectángulo?

Porque en programación **un tipo se define por su comportamiento (contrato), no por su definición matemática**. Un `Rectangulo` *mutable* promete que `set_ancho()` cambia **solo** el ancho y `set_alto()` **solo** el alto, por lo que después de `set_ancho(4)` y `set_alto(5)` el área será 20.

`Cuadrado` rompe esa promesa: al cambiar un lado modifica también el otro (efecto lateral). El resultado del test fue **área = 25 en vez de 20**. Como `Cuadrado` **no puede sustituir** a `Rectangulo` sin alterar el resultado, la herencia es incorrecta.

En términos de Liskov: `Cuadrado` **fortalece una invariante** (ancho = alto) que la base no tenía, y por tanto cambia las poscondiciones de `set_ancho` / `set_alto`.

5.5 Alternativa de diseño propuesta

Se propone que `Rectangulo` y `Cuadrado` sean **clases hermanas** bajo una abstracción común `FiguraGeometrica` (con `area()` y `perimetro()`), y que además sean **inmutables**. Así no existen `set_ancho` / `set_alto` que puedan contradecirse: si se quiere otro ancho, se crea un objeto nuevo (`con_ancho(v)`). Otras opciones válidas son usar **composición** (un `Cuadrado` que internamente contiene un `Rectangulo`) o una interfaz independiente.

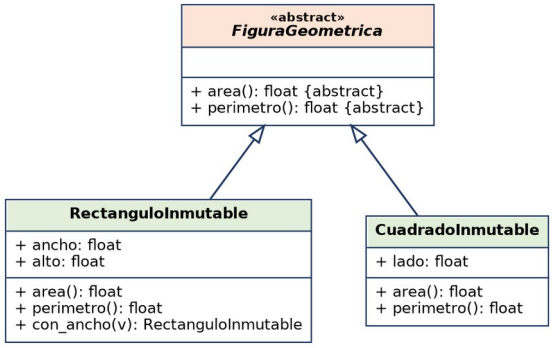


Figura 3. Rediseño: hermanos bajo una abstracción, sin herencia Cuadrado → Rectangulo.

Rediseño que cumple el LSP

```
@dataclass(frozen=True)
class RectanguloInmutable(FiguraGeometrica):
    ancho: float
    alto: float

    def area(self) -> float:
        return self.ancho * self.alto

    def perimetro(self) -> float:
        return 2 * (self.ancho + self.alto)

    def con_ancho(self, v: float) -> "RectanguloInmutable":
        return RectanguloInmutable(v, self.alto)  # devuelve un objeto NUEVO


@dataclass(frozen=True)
class CuadradoInmutable(FiguraGeometrica):
    lado: float

    def area(self) -> float:
        return self.lado ** 2

    def perimetro(self) -> float:
        return 4 * self.lado
```

Análisis

Parte A: las 4 subclases de personal pasan el contrato → cumplen el LSP.

Parte B: el mismo test que pasa con `Rectangulo` falla con `Cuadrado` → **violación demostrada con evidencia** (25 ≠ 20).

Parte C: con el rediseño, cualquier `FiguraGeometrica` es intercambiable y el contrato se cumple en ambas.

6. Actividad 4 — Jerarquía en C++17 (virtual, override y destructor)

Qué se pide: implementar en C++17 la jerarquía equivalente a la Actividad 2, con **destructor virtual** en la base, **método virtual puro** `calcularRemuneracion()`, polimorfismo con `vector<unique_ptr<...>>` y la función `planilla()`. Archivos: `actividad4_cpp/personal_unap.hpp` y `main.cpp`.

6.1 Implementación

personal_unap.hpp — clase base (virtual puro + destructor virtual) y Docente

```
class PersonalUniversitario {
protected:
    std::string dni_;
    std::string nombres_;
    std::string apellidos_;

    static std::string validarDni(std::string dni) {
        if (dni.size() != 8 ||
            dni.find_first_not_of("0123456789") != std::string::npos)
            throw std::invalid_argument("DNI invalido: '" + dni +
                                        "' (debe tener 8 digitos)");
        return dni;
    }

public:
    PersonalUniversitario(std::string dni, std::string nombres,
                          std::string apellidos)
        : dni_(validarDni(std::move(dni))),
          nombres_(std::move(nombres)),
          apellidos_(std::move(apellidos)) {}

    virtual ~PersonalUniversitario() = default;  // destructor virtual OBLIGATORIO

    virtual float calcularRemuneracion() const = 0;  // método virtual puro

    virtual std::string identificarse() const {
        return apellidos_ + ", " + nombres_ + " (DNI: " + dni_ + ")";
    }
};

class Docente : public PersonalUniversitario {
    int horas_;
    float tarifa_ = 75.0f;
    float factor_;

    static float resolverFactor(const std::string& cat) {
        if (cat == "Auxiliar") return 1.0f;
        if (cat == "Asociado") return 1.4f;
        if (cat == "Principal") return 1.8f;
        throw std::invalid_argument("Categoria invalida: " + cat);
    }

public:
    Docente(std::string dni, std::string nombres, std::string apellidos,
            int horas, const std::string& categoria)
        : PersonalUniversitario(std::move(dni), std::move(nombres),
                                std::move(apellidos)),
          horas_(horas), factor_(resolverFactor(categoria)) {
        if (horas <= 0 || horas > 40)
            throw std::invalid_argument("Horas fuera de rango [1,40]");
```

```
    }

    float calcularRemuneracion() const override {
        return horas_ * tarifa_ * factor_;
    }
};
```

personal_unap.hpp — planilla()

```
inline void planilla(const std::vector<std::unique_ptr<PersonalUniversitario>>& p) {
    float total = 0.0f;
    std::cout << std::string(60, '=') << "\n";
    std::cout << "PLANILLA PERSONAL UNIVERSITARIO — UNAP PUNO\n";
    std::cout << std::string(60, '=') << "\n";
    for (const auto& persona : p) {
        std::cout << std::fixed << std::setprecision(2)
                  << persona->identificarse()
                  << " S/ " << persona->calcularRemuneracion() << "\n";
        total += persona->calcularRemuneracion();
    }
    std::cout << std::string(60, '-') << "\n";
    std::cout << "TOTAL: S/ " << total << "\n";
}
```

6.2 Compilación y salida

Compilación con -Wall -Wextra -pedantic

```
$ make
g++ -std=c++17 -Wall -Wextra -pedantic -o personal main.cpp
(0 errores, 0 warnings)
```

$ ./personal

```
============================================================
PLANILLA PERSONAL UNIVERSITARIO — UNAP PUNO
============================================================
Mamani Quispe, Juan Carlos (DNI: 12345678) S/ 1680.00
Quispe Torres, Maria Elena (DNI: 23456789) S/ 1620.00
Huanca Lazo, Pedro (DNI: 34567890) S/ 3150.00
Flores Ramos, Ana Lucia (DNI: 45678901) S/ 3000.00
------------------------------------------------------------
TOTAL: S/ 9450.00

PRUEBA DE VALIDACIONES
------------------------------------------------------------
[OK] DNI con 7 digitos
     -> invalid_argument: DNI invalido: '1234567' (debe tener 8 digitos)
[OK] Horas = 50 (max. 40)
     -> invalid_argument: Horas fuera de rango [1,40]
[OK] Categoria inexistente
     -> invalid_argument: Categoria invalida: Titular
[OK] Salario base negativo
     -> invalid_argument: Salario base debe ser positivo
[OK] Investigador sin proyectos
     -> invalid_argument: Debe tener al menos 1 proyecto activo
```

6.3 Análisis

- **virtual ... = 0** convierte a `PersonalUniversitario` en clase abstracta: no se puede instanciar y obliga a cada subclase a implementar `calcularRemuneracion()`.
- **override** hace que el compilador verifique que realmente se sobrescribe un método virtual; si hubiera un error de firma (p. ej. olvidar `const`), no compilaría.

- **unique_ptr + destructor virtual:** al salir de `main`, cada `unique_ptr<PersonalUniversitario>` destruye el objeto real correctamente, sin fugas de memoria.
- **Mismo resultado que Python:** total **S/ 9450.00**, lo que confirma que ambas implementaciones son equivalentes.
- **Mejoras sobre el enunciado:** `planilla()` se declaró `inline` (evita errores de definición múltiple si el header se incluye en varios `.cpp`), se agregó la clase `Investigador`, se validó el DNI y se incluyó `<utility>` para `std::move`.

7. Actividad 5 — Proyecto Integrador: Sistema de Biblioteca

Qué se pide: aplicar herencia al Proyecto Integrador: una clase base y al menos dos subclases IS-A, implementadas en Python y C++17, con un método sobrescrito por subclase, una función que demuestre polimorfismo sobre una lista heterogénea, UML actualizado y commit.

> **Dominio elegido**
> Sistema de **Biblioteca Universitaria UNAP**. La clase base `RecursoBibliografico` modela todo material que se presta; sus subclases `LibroFisico`, `LibroDigital` y `Revista` sobrescriben el plazo de préstamo, el cálculo de multa y la descripción.

7.1 Tabla de relaciones IS-A verificadas

Clase base	Subclase 1	Subclase 2	Subclase 3 (extra)
`RecursoBibliografico` (abstracta)	`LibroFisico`	`LibroDigital`	`Revista`
Relación IS-A verificada	Un libro físico es un recurso bibliográfico ✔️	Un libro digital es un recurso bibliográfico ✔️	Una revista es un recurso bibliográfico ✔️
Método(s) sobrescrito(s)	`dias_prestamo` = 7 `calcular_multa` = S/ 1.50/día	`dias_prestamo` = 14 `calcular_multa` = 0 (caduca solo)	`dias_prestamo` = 3 `calcular_multa` = S/ 2.00/día
Prueba LSP	Contrato cumplido ✔️	Contrato cumplido ✔️	Contrato cumplido ✔️

7.2 Diagrama de clases UML actualizado

El diagrama muestra la **herencia** (flecha hueca ▷) y las relaciones de **composición** (rombo lleno ◆: la `Biblioteca` es dueña de su catálogo) y **asociación** (línea punteada: un `Prestamo` referencia a un `Usuario` y a un `RecursoBibliografico`).

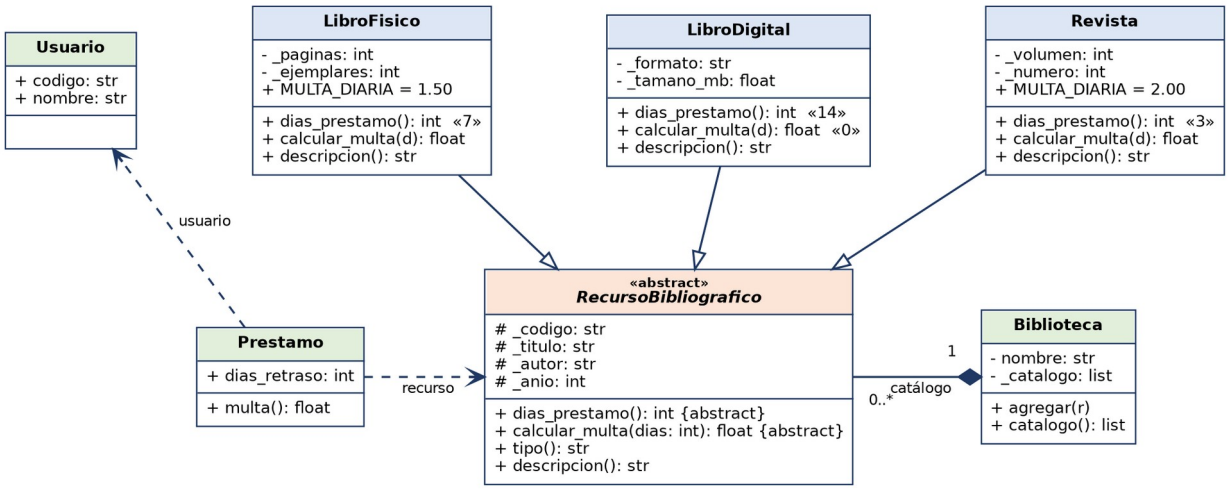


Figura 4. UML del Proyecto Integrador: herencia + composición + asociación.

7.3 Decisiones de diseño: herencia vs composición

- `Prestamo` **no** hereda de `RecursoBibliografico`: un préstamo no *es* un recurso, lo *referencia* → **asociación**.
- `Biblioteca` **no** hereda de `list`: *contiene* un catálogo → **composición** (`unique_ptr` en C++; copia defensiva en Python).
- `LibroDigital` **no** hereda de `LibroFisico` aunque "ambos son libros": un libro digital no tiene páginas físicas ni ejemplares. Por eso ambos son **hermanos** bajo `RecursoBibliografico`.

7.4 Implementación en Python

biblioteca.py — base abstracta y LibroFisico

```
class RecursoBibliografico(ABC):
    """Clase base abstracta de todo material que presta la biblioteca."""

    def __init__(self, codigo: str, titulo: str, autor: str, anio: int):
        if not codigo:
            raise ValueError("El código no puede estar vacío")
        if anio < 1450:
            raise ValueError(f"Año inválido: {anio}")
        self._codigo = codigo
        self._titulo = titulo
        self._autor = autor
        self._anio = anio

    @property
    def codigo(self) -> str:
        return self._codigo

    @property
    def titulo(self) -> str:
        return self._titulo

    @abstractmethod
    def dias_prestamo(self) -> int:
        """Días máximos que el usuario puede tener el recurso."""

    @abstractmethod
    def calcular_multa(self, dias_retraso: int) -> float:
        """Multa en S/ por devolver con 'dias_retraso' días de atraso (>= 0)."""

    def tipo(self) -> str:
        return "Recurso"

    def descripcion(self) -> str:
        return f"[{self.tipo()}] {self._titulo} — {self._autor} ({self._anio})"

    def __str__(self) -> str:
        return self.descripcion()


class LibroFisico(RecursoBibliografico):
    MULTA_DIARIA = 1.50

    def __init__(self, codigo: str, titulo: str, autor: str, anio: int,
                 paginas: int, ejemplares: int = 1):
        super().__init__(codigo, titulo, autor, anio)
        if paginas <= 0 or ejemplares < 1:
            raise ValueError("Páginas y ejemplares deben ser positivos")
```

```
        self._paginas = paginas
        self._ejemplares = ejemplares

    @override
    def tipo(self) -> str:
        return "Libro físico"

    @override
    def dias_prestamo(self) -> int:
        return 7

    @override
    def calcular_multa(self, dias_retraso: int) -> float:
        return max(0, dias_retraso) * self.MULTA_DIARIA

    @override
    def descripcion(self) -> str:
        return (f"{super().descripcion()} · {self._paginas} págs · "
                f"{self._ejemplares} ejemplar(es)")
```

biblioteca.py — LibroDigital

```
class LibroDigital(RecursoBibliografico):
    """El préstamo digital expira solo: no genera multa por retraso."""

    def __init__(self, codigo: str, titulo: str, autor: str, anio: int,
                 formato: str, tamano_mb: float):
        super().__init__(codigo, titulo, autor, anio)
        if formato.upper() not in {"PDF", "EPUB"}:
            raise ValueError(f"Formato no soportado: {formato}")
        if tamano_mb <= 0:
            raise ValueError("El tamaño debe ser positivo")
        self._formato = formato.upper()
        self._tamano_mb = tamano_mb

    @override
    def tipo(self) -> str:
        return "Libro digital"

    @override
    def dias_prestamo(self) -> int:
        return 14

    @override
    def calcular_multa(self, dias_retraso: int) -> float:
        return 0.0  # la licencia caduca automáticamente → contrato (>= 0) se cumple

    @override
    def descripcion(self) -> str:
        return (f"{super().descripcion()} · {self._formato} · "
                f"{self._tamano_mb:.1f} MB")
```

biblioteca.py — polimorfismo sobre lista heterogénea (sin isinstance)

```
def reporte_catalogo(recursos: list[RecursoBibliografico],
                     dias_retraso: int = 4) -> None:
    """Itera la lista heterogénea SIN usar isinstance()."""
    print("=" * 78)
    print(f"CATÁLOGO — multa simulada con {dias_retraso} días de retraso")
    print("=" * 78)
    total = 0.0
    for r in recursos:
        multa = r.calcular_multa(dias_retraso)  # ← método distinto por subclase
```

```
        total += multa
        print(r.descripcion())
        print(f"    Préstamo: {r.dias_prestamo():>2} días │ Multa: S/ {multa:>5.2f}")
    print("-" * 78)
    print(f"TOTAL MULTAS SIMULADAS: S/ {total:.2f}")
```

7.5 Implementación en C++17

biblioteca.hpp — base abstracta con virtual puro y destructor virtual

```
class RecursoBibliografico {
protected:
    std::string codigo_;
    std::string titulo_;
    std::string autor_;
    int anio_;

public:
    RecursoBibliografico(std::string codigo, std::string titulo,
                         std::string autor, int anio)
        : codigo_(std::move(codigo)), titulo_(std::move(titulo)),
          autor_(std::move(autor)), anio_(anio) {
        if (codigo_.empty())
            throw std::invalid_argument("El codigo no puede estar vacio");
        if (anio_ < 1450)
            throw std::invalid_argument("Anio invalido: " + std::to_string(anio_));
    }

    virtual ~RecursoBibliografico() = default;  // destructor virtual OBLIGATORIO

    const std::string& codigo() const { return codigo_; }
    const std::string& titulo() const { return titulo_; }

    virtual int diasPrestamo() const = 0;                    // virtual puro
    virtual double calcularMulta(int diasRetraso) const = 0;  // virtual puro
    virtual std::string tipo() const { return "Recurso"; }

    virtual std::string descripcion() const {
        return "[" + tipo() + "] " + titulo_ + " - " + autor_ + " (" +
               std::to_string(anio_) + ")";
    }
};
```

biblioteca.hpp — LibroFisico con override

```
class LibroFisico : public RecursoBibliografico {
    static constexpr double MULTA_DIARIA = 1.50;
    int paginas_;
    int ejemplares_;

public:
    LibroFisico(std::string codigo, std::string titulo, std::string autor,
                int anio, int paginas, int ejemplares = 1)
        : RecursoBibliografico(std::move(codigo), std::move(titulo),
                               std::move(autor), anio),
          paginas_(paginas), ejemplares_(ejemplares) {
        if (paginas_ <= 0 || ejemplares_ < 1)
            throw std::invalid_argument("Paginas y ejemplares deben ser positivos");
    }

    std::string tipo() const override { return "Libro fisico"; }
    int diasPrestamo() const override { return 7; }
```

```
    double calcularMulta(int diasRetraso) const override {
        return std::max(0, diasRetraso) * MULTA_DIARIA;
    }
    std::string descripcion() const override {
        return RecursoBibliografico::descripcion() + " | " +
               std::to_string(paginas_) + " pags | " +
               std::to_string(ejemplares_) + " ejemplar(es)";
    }
};
```

biblioteca.hpp — reporteCatalogo(): despacho dinámico por vtable

```
inline void reporteCatalogo(
    const std::vector<std::unique_ptr<RecursoBibliografico>>& recursos,
    int diasRetraso) {
    std::cout << std::string(78, '=') << "\n"
              << "CATALOGO - multa simulada con " << diasRetraso
              << " dias de retraso\n"
              << std::string(78, '=') << "\n";
    double total = 0.0;
    for (const auto& r : recursos) {
        double multa = r->calcularMulta(diasRetraso);  // despacho por vtable
        total += multa;
        std::cout << r->descripcion() << "\n"
                  << "    Prestamo: " << std::setw(2) << r->diasPrestamo()
                  << " dias | Multa: S/ " << std::fixed << std::setprecision(2)
                  << std::setw(5) << multa << "\n";
    }
    std::cout << std::string(78, '-') << "\n"
              << "TOTAL MULTAS SIMULADAS: S/ " << total << "\n";
}
```

7.6 Salidas de los programas

$ python3 actividad5_proyecto/python/biblioteca.py

```
================================================================================
CATÁLOGO — multa simulada con 4 días de retraso
================================================================================
[Libro físico] Clean Code — Robert C. Martin (2008) · 464 págs · 3 ejemplar(es)
    Préstamo:  7 días | Multa: S/  6.00
[Libro digital] Design Patterns — Gamma et al. (1994) · PDF · 12.5 MB
    Préstamo: 14 días | Multa: S/  0.00
[Revista] Revista de Ingeniería de Sistemas — UNAP (2025) · Vol. 12 Nº 2
    Préstamo:  3 días | Multa: S/  8.00
--------------------------------------------------------------------------------
TOTAL MULTAS SIMULADAS: S/ 14.00

Verificación LSP: OK — las 3 subclases sustituyen a RecursoBibliografico

PRÉSTAMO (asociación + delegación polimórfica)
--------------------------------------------------------------------------------
Kevin Manuel Vilca Paco devolvió 'Clean Code' con 2 día(s) de retraso → multa S/ 3.00
Kevin Manuel Vilca Paco devolvió 'Design Patterns' con 10 día(s) de retraso → multa S/ 0.00
Kevin Manuel Vilca Paco devolvió 'Revista de Ingeniería de Sistemas' con 1 día(s) de retraso →
multa S/ 2.00

VALIDACIÓN
--------------------------------------------------------------------------------
[OK] Código duplicado: RV-001
```

$ cd actividad5_proyecto/cpp && make run

```
================================================================================
```

```
CATALOGO - multa simulada con 4 dias de retraso
================================================================================
[Libro fisico] Clean Code - Robert C. Martin (2008) | 464 pags | 3 ejemplar(es)
    Prestamo:  7 dias | Multa: S/  6.00
[Libro digital] Design Patterns - Gamma et al. (1994) | PDF | 12.5 MB
    Prestamo: 14 dias | Multa: S/  0.00
[Revista] Revista de Ingenieria de Sistemas - UNAP (2025) | Vol. 12 No. 2
    Prestamo:  3 dias | Multa: S/  8.00
--------------------------------------------------------------------------------
TOTAL MULTAS SIMULADAS: S/ 14.00

Verificacion LSP: OK - las 3 subclases sustituyen a RecursoBibliografico

VALIDACION
--------------------------------------------------------------------------------
[OK] Codigo duplicado: RV-001
```

7.7 Análisis

- **Polimorfismo demostrado:** `reporte_catalogo()` / `reporteCatalogo()` recorre una lista con tres tipos distintos y obtiene multas diferentes con **la misma llamada** `calcular_multa(4)`: S/ 6.00, S/ 0.00 y S/ 8.00.
- **LSP:** `LibroDigital` devuelve multa 0, que sigue cumpliendo el contrato de la base (multa ≥ 0, sin retraso = 0). No lanza excepciones ni cambia el significado del método.
- **Equivalencia Python / C++:** ambos programas producen el mismo resultado (total de multas simuladas **S/ 14.00**).

8. Preguntas de reflexión

Pregunta 1 — ¿Herencia o composición en el Proyecto Integrador?

Sí, hubo dos casos. **(1)** Al inicio pensé que `LibroDigital` podía heredar de `LibroFisico` para reutilizar título y autor; sin embargo, un libro digital no tiene páginas físicas ni ejemplares, así que habría heredado atributos que no le corresponden y rompería el LSP. Elegí una base más general (`RecursoBibliografico`) con ambos como hermanos. **(2)** También consideré que `Prestamo` heredara de `RecursoBibliografico` para acceder a sus datos, pero un préstamo no *es* un recurso: lo *usa*. Elegí **asociación** (el préstamo guarda una referencia al recurso y al usuario) y delega `calcular_multa()` al recurso.

Pregunta 2 — ¿Por qué el destructor de la base debe ser virtual en C++?

Cuando se usa polimorfismo se suele destruir un objeto a través de un puntero a la base (por ejemplo, un `unique_ptr<PersonalUniversitario>` que apunta a un `Docente`). Si el destructor **no** es virtual, la llamada `delete ptr_base` no consulta la vtable: el compilador solo sabe que el tipo estático es la base y ejecuta **únicamente el destructor de la base**. Según el estándar esto es **comportamiento indefinido**; en la práctica **no se ejecuta el destructor de la subclase**, por lo que los recursos que ella administraba (memoria dinámica, archivos, conexiones) **se pierden (fuga de recursos)**. Con `virtual ~Base() = default;` la destrucción se resuelve dinámicamente: primero se ejecuta el destructor de la subclase real y luego el de la base, en el orden correcto.

Pregunta 3 — Duck typing (Python) vs tipado estricto (C++)

	Python (duck typing)	C++ (tipos estrictos + virtual)
Ventajas	• Flexible: sirve cualquier objeto con el método correcto. • Menos código repetitivo. • Fácil de probar con objetos falsos (mocks). • Clases independientes, sin acoplarse a una base.	• Errores detectados al compilar. • Contrato explícito y verificable (`override`, `= 0`). • Mejor rendimiento y herramientas de análisis. • Documentación clara de la jerarquía.
Desventajas	• Errores solo aparecen al ejecutar (`AttributeError`). • El contrato es implícito. • Más difícil de analizar con herramientas.	• Rígido: obliga a heredar de una base común. • Más código (headers, virtual, override). • Mayor acoplamiento y pequeño costo de vtable.

Punto medio en Python: las clases abstractas (`ABC`), los `Protocol` y las anotaciones de tipos permiten tener la flexibilidad de Python con parte de la seguridad de C++. En esta práctica se usó `ABC` + `@override` para obtener ambas ventajas.

9. Conclusiones

- La herencia solo se justifica cuando existe una relación **IS-A real y sustituible**; en caso contrario, la composición o la asociación producen diseños más simples y flexibles.
- El **polimorfismo dinámico** permitió escribir funciones (`planilla`, `reporte_catalogo`) que funcionan con cualquier subclase presente o **futura**, sin `isinstance()` ni `if/else` por tipo.
- El **LSP** se comprueba con **contratos ejecutables**: el mismo test debe pasar para la base y para todas sus subclases. El caso Cuadrado/Rectángulo demostró que "ser un" matemáticamente no implica ser un subtipo.
- En C++ son indispensables el **destructor virtual**, `override` y `unique_ptr`; en Python, `ABC`, `@override` y `super()` ofrecen garantías equivalentes.
- Ambos lenguajes produjeron resultados idénticos (S/ 9450.00 en la planilla; S/ 14.00 en las multas simuladas), validando la equivalencia de las implementaciones.

Resumen de resultados

Act.	Contenido	Resultado	Estado
1	Análisis IS-A vs HAS-A	4 herencia · 2 composición	✔️ Completo
2	Jerarquía de personal (Python)	Total S/ 9450.00 · 6 validaciones OK	✔️ Ejecuta sin errores
3	LSP: contrato + Cuadrado	Violación demostrada (25 ≠ 20) + rediseño	✔️ Completo
4	Jerarquía en C++17	Total S/ 9450.00 · 0 warnings	✔️ Compila `-Wall -Wextra`
5	Proyecto Integrador (Biblioteca)	1 base + 3 subclases · Python + C++ · UML	✔️ Completo

10. Referencias

cppreference.com. (2026). virtual function specifier. https://en.cppreference.com/w/cpp/language/virtual

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns. Addison-Wesley.

Larman, C. (2004). Applying UML and Patterns (3rd ed.). Prentice Hall.

Liskov, B. (1987). Data abstraction and hierarchy. ACM SIGPLAN Notices, 23(5). https://doi.org/10.1145/62138.62141

Martin, R. C. (2008). Clean Code. Prentice Hall.

Zanabria Gálvez, A. H. (2026). Guía de Práctica de Laboratorio — Semana 3: Herencia y Polimorfismo. SIS206 POO II. Universidad Nacional del Altiplano, Puno.

Anexo — Estructura del repositorio y ejecución

Árbol de carpetas

```
Practica_Semana3_POO2_Kevin_Vilca/
├── README.md
├── informe/                    ← este informe (Word y PDF)
├── actividad1_isa_vs_hasa/     analisis.md
├── actividad2_python/          personal_unap.py
├── actividad3_lsp/             lsp_verificacion.py
├── actividad4_cpp/             personal_unap.hpp · main.cpp · Makefile
├── actividad5_proyecto/
│   ├── python/                 biblioteca.py
│   ├── cpp/                    biblioteca.hpp · main.cpp · Makefile
│   └── uml/                    uml_biblioteca.png / .dot
├── uml/                        diagramas PNG + generar_uml.py
└── salidas/                    salida real de cada programa
```

Ejecución

```
# Python 3.12+
python3 actividad2_python/personal_unap.py
python3 actividad3_lsp/lsp_verificacion.py
python3 actividad5_proyecto/python/biblioteca.py

# C++17
cd actividad4_cpp && make run
cd ../actividad5_proyecto/cpp && make run
```

Subir a GitHub

```
git init
git add .
git commit -m "feat(semana3): jerarquía de herencia y polimorfismo"
git branch -M main
git remote add origin https://github.com/<usuario>/<repositorio>.git
git push -u origin main
```